

第3章 表、栈和队列

建议学时:4-5 小时 | 难度:★★★☆☆ | 读者背景:已掌握 C 语言,DS&A 零基础

🎯 本章学习目标

- 理解**抽象数据类型(ADT)**思想:先定义"做什么",再谈"怎么实现"
- 掌握表的两类实现:数组实现(倍增扩容)与链式实现(结点+引用),会对比两者复杂度
- 完成从 C 的 `malloc` 数组到 Python `list` 的迁移,理解"list 就是动态数组"
- 能说清 Python 中为什么**几乎不需要手写链表**,以及什么时候仍然要写
- 能独立实现栈的数组版与链式版,并用栈解决括号平衡、后缀求值、中缀转后缀三个经典问题
- 能独立实现循环数组队列,说清空/满判断的两种方案
- 学完能独立实现中缀表达式计算器(栈应用综合),并会正确使用 `list` 模拟栈与 `collections.deque` 队列

💡 从 C 到 Python:本章主题如何迁移

本章是全书第一个"真正动手实现数据结构"的章节。算法直觉先给结论:表、栈、队列是后续所有复杂结构的积木——树的层序遍历要用队列,图的 DFS 要用栈,第6章的堆本质是数组,第5章的散列表是"数组+链表"的合体。本章学不透,后面每一章都会还债。

在 C 里,你已经会两种原始工具:`malloc` 出来的动态数组和 `struct` + 指针串起来的链表。C 的痛点是双重的:动态数组要自己管理"容量、长度、扩容"三件事,一次 `realloc` 写错就是内存泄漏或堆损坏;链表没有语言支持,每个操作都要手写指针舞蹈,一不小心就访问悬垂指针。

Python 的解法是**把动态数组直接内建成语言的一部分**:`list` 就是自动扩容的动态数组,`append` 平均 $O(1)$,垃圾回收器替你释放内存——C 里最头疼的"容量、长度、扩容、释放"四件事全部消失。这是巨大的解放,但有两个代价:① 没有"裸数组"让你看到底层,扩容细节藏在解释器里(\$3.3.1 会手写一遍把原理补回来);② `list` 的元素是指向对象的引用,与 C 的"连续存放的 `int`"不同——不过对本章内容影响不大。

链表在 Python 里地位特殊:**几乎不需要手写**——因为 `list` 太方便,而且链表的 $O(1)$ 头插优势 Python 有 `deque` 承担。但"链表"作为概念仍然必须掌握:第4章的树、第5章的分离链接散列表都建立在"结点+引用"之上。本章把链表当作"指针思维的训练场",Python 版用"对象引用"替代指针。

栈与队列部分,本章用三个经典应用(括号平衡、后缀求值、中缀转后缀)演示"数据结构服务于算法"的思维方式——它们是编译原理与表达式求值的原型,也是各大面试的高频题。

📦 前置知识自查

数学前置:本章几乎不需要新数学;倍增扩容的摊还分析直觉即可(严格分析在第11章)。

C 语言前置:`malloc` / `free` 动态数组、`struct` 与 `->` 指针访问、`char*` 字符串遍历。

依赖的前面章节:第1章递归(栈的递归实现对照)、第2章大 O 记号(本章复杂度速查表直接用)。

🚀 本章学习路线

1. **第一阶段(1 小时)**:读 §3.1–§3.2,建立 ADT 概念,在 REPL 里实验 list 的 append/insert/pop
2. **第二阶段(1.5 小时)**:读 §3.3–§3.4,手写动态数组与单链表,上机验证
3. **第三阶段(1.5 小时)**:读 §3.5–§3.6,手写栈与队列,完成三个栈应用例题
4. **第四阶段(实验,1 小时)**:完成章末实验中缀表达式计算器
5. **验收标准**:能默写单链表插入/删除、循环队列、中缀转后缀,并说清每步复杂度

本章知识导图

- §3.1 ADT:抽象数据类型(接口与实现分离、从 C 的 struct+函数指针到类)
- §3.2 表 ADT 与从 C 的迁移(malloc 数组 → list、倍增扩容、list 的底层真相)
- §3.3 表的数组实现(手写动态数组、list 常用操作、切片与遍历)
- §3.4 表的链式实现(结点与引用、手写单链表、“几乎不手写链表”的工程判断)
- §3.5 栈(list 模拟、括号平衡、后缀求值、中缀转后缀)
- §3.6 队列(循环数组实现、空/满判断的两种方案、collections.deque)
- §3.7 选型与工程实践(何时数组何时链表、栈模拟递归、调试技巧)

§3.1 ADT:抽象数据类型

3.1.1 什么是 ADT

抽象数据类型(ADT) = 一组操作 + 这些操作的语义, **不涉及实现细节**。例如“栈”这个 ADT 只有三个操作:push(入栈)、pop(出栈)、top(取栈顶),语义是“后进先出(LIFO)”。至于底层用数组还是链表、满时怎么办,属于“实现”,与 ADT 无关。
好处: 使用者只依赖接口,实现可以随时替换——把数组栈换成链式栈,使用方代码一行不用改。

3.1.2 从 C 的 struct+函数指针到 Python 的类

C 语言没有类,模拟 ADT 只能用 struct 存数据 + 函数指针表存操作:

C 的写法(struct + 函数指针)	Python 的写法(class 封装)
<pre>struct Stack { int *a; int top; int cap; };</pre>	<pre>class Stack: def __init__(self): self.a = []</pre>
<pre>stack_init(&s); stack_push(&s, x);</pre> 每个操作传 &s	s.push(x) 数据与操作天然绑定
使用者能“意外”访问 s.a[0] 破坏结构	self.a 靠约定保护(_ 前缀,Python 无强制 private)
忘记 stack_destroy 即泄漏,没有提醒	垃圾回收器自动释放,无需操心

结论:ADT 的思想两种语言都能表达,Python 的类是它的自然载体——本书所有数据结构都以“类”的形式呈现。

§3.2 表 ADT 与从 C 的迁移

3.2.1 表的基本操作

表(list)ADT:线性有序元素的集合,常用操作——按下标访问、按值查找、任意位置插入/删除、遍历。**同一个 ADT,两种截然不同的实现**(数组与链表),各有胜负,本章的主线就是这张对照表:

操作	数组实现(list)	链式实现
按下标访问	$O(1)$	$O(n)$
按值查找	$O(n)$	$O(n)$
尾部插入	平均 $O(1)$	$O(1)$
头部插入	$O(n)$	$O(1)$
任意位置插入/删除	$O(n)$ (移动元素)	$O(1)$ (已定位后)

3.2.2 C 的 malloc 数组 → Python 的 list

回忆 C 里管理动态数组的三件套:记录长度、记录容量、`realloc` 扩容。Python 的 `list` 把这套动作全部内置: `len(a)` 是长度;容量没有直接接口(可用 `sys.getsizeof` 侧面观察);扩容自动发生,策略是**按比例增长**(CPython 实现约 1/8 增量,思想与倍增相同)。

C 的写法	Python 的写法
<code>int *a = malloc(n * sizeof(int));</code>	<code>a = [0] * n</code> 或 <code>a = []</code>
<code>a[len++] = x;</code> (自己管长度)	<code>a.append(x)</code>
<code>realloc</code> 手动扩容(易错)	自动扩容,解释器管理
<code>free(a);</code> (忘记即泄漏)	垃圾回收器自动释放
<code>a[i]</code> 越界 = 未定义行为(可能崩溃)	<code>a[i]</code> 越界抛 <code>IndexError</code> ,显式暴露错误

倍增扩容:当 `append` 时容量已满,解释器分配一块更大的新内存,拷贝全部旧元素的引用,释放旧内存。直觉上“每次扩容都是 $O(n)$,那插入怎么还说平均 $O(1)$?”——因为扩容发生得越来越少:容量从 1 倍增到 n 需要 $\log_2 n$ 次扩容,总拷贝代价 $n + n/2 + n/4 + \dots \approx 2n$,摊到 n 次插入上平均每次 $O(1)$ 。严格的摊还分析见第11章,现在先记住结论: **倍增使得尾部插入平均 $O(1)$** 。

⚠ 误区 1:把 list 当 C 数组用

两个迁移坑:① `a[i] = x` 不会自动扩容——`a = []` 后直接 `a[0] = 1` 抛 `IndexError` (与 C 的未定义行为不同,Python 显式报错,这反而是好事);② `a = [0] * 5` 中的元素是**同一个对象 0** 的 5 份引用,对不可变对象无感,但写 `m = [[]] * 5` 会得到 5 个**共享**的空列表——`m[0].append(1)` 后 `m[1]` 也变了。要独立列表用 `[[] for _ in range(5)]`。

§3.3 表的数组实现

3.3.1 手写动态数组:搞懂倍增

示例 3-1 手写动态数组:倍增扩容

```
# 手写动态数组:容量不足时倍增(2 倍)拷贝
# 工程中直接用 list 即可,这里手写是为了看清 append 的底层
class MyVector:
    def __init__(self):
        self._data = [None] * 1      # 底层存储,初始容量 1
        self._size = 0               # 元素个数(长度)
        self._cap = 1               # 已分配容量

    def _grow(self):                 # 倍增:新容量 = 旧容量 × 2
        new_cap = self._cap * 2
        new_data = [None] * new_cap
        for i in range(self._size):
            new_data[i] = self._data[i]  # 拷贝旧元素
        self._data = new_data
        self._cap = new_cap

    def push_back(self, x):
        if self._size == self._cap: # 满了:扩容
            self._grow()
        self._data[self._size] = x
        self._size += 1

    def __getitem__(self, i):        # 支持 v[i] 下标访问
        return self._data[i]

    def __len__(self):               # 支持 len(v)
        return self._size

    @property
    def capacity(self):
        return self._cap
```

3.3.2 list 常用操作速览

示例 3-2 list 核心操作

```
v = []                # 空表
v.append(3)           # 尾部插入:3
v.append(1)           # 3 1
v.insert(0, 9)        # 头部插入:9 3 1(O(n),搬移全部元素)
v.pop(1)              # 删除下标 1:9 1(O(n))
v[1] = 5              # 随机访问 O(1):9 5
for x in v:
    print(x, end=" ") # 遍历
print(f"\nsize={len(v)}") # 长度(容量由解释器管理,无直接接口)
```

3.3.3 切片:Python 独有的"批量视图"

切片 `a[i:j]` 取下标 `i` 到 `j-1` 的**新列表**(拷贝引用)。它是 C 没有的利器,但有两个复杂度陷阱:① `a[i:j]` 本身 $O(j-i)$, 写在循环里可能把 $O(n)$ 变 $O(n^2)$ (第2章算法一正是栽在这);② `a[:]` 是浅拷贝——元素是可变对象时,内层对象仍共享。删除与替换也有切片语法: `del a[1:3]`、`a[1:1] = [9, 8]` (插入)。

§3.4 表的链式实现

3.4.1 结点与引用

链表的原子单位是**结点(Node)**:一个数据域 + 一个指向下一个结点的引用。C 的 `struct Node { int val; struct Node *next; }` 在 Python 里变成:

示例 3-3 结点类:C 的 `struct`+指针 → Python 的 `class`+引用

```
class Node:
    def __init__(self, val, nxt=None):
        self.val = val          # 数据域
        self.next = nxt        # 指向下一个结点(引用即指针)

# C 里 Node *p = malloc(sizeof(Node)); p->val = 1;
# Python 里:
p = Node(1)
q = Node(2, p)                # q.next 指向 p
print(q.next.val)             # 1 — 与 C 的 q->next->val 同义
```

Python 的引用与 C 的指针本质相同("指向对象"),区别是:引用不能做算术、不能指向"未初始化的内存",且对象引用计数归零时自动回收——C 的悬垂指针与内存泄漏两大病根,Python 从机制上消灭。

3.4.2 手写单链表

示例 3-4 单链表:表头插入、查找、删除

```
def insert_front(head, x):      # 表头插入 O(1),返回新表头
    return Node(x, head)

def find(head, x):              # 查找 O(n)
    while head is not None and head.val != x:
        head = head.next
    return head                  # 找不到返回 None

def erase(head, x):              # 删除值为 x 的第一个结点,返回新表头
    if head is None:
        return None
    if head.val == x:            # 删表头是特例
        return head.next        # 旧表头失去引用,自动回收(无需 delete!)
    prev = head                  # 其余情况:找前驱
    while prev.next is not None and prev.next.val != x:
        prev = prev.next
    if prev.next is not None:
        prev.next = prev.next.next # 前驱跳过目标结点
    return head
```

★ 重点:插入删除的"定位"成本

说"链表插入删除 $O(1)$ "是有前提的: 已经定位到操作位置。定位本身(查找值、走到第 k 个结点)是 $O(n)$ 。所以"链表的插入比数组快"只在特定场景成立: 频繁头部操作、或已持有位置引用的遍历过程中。速查表里的" $O(1)$ "指的都是"已定位后"。

🚀 工程建议:Python 里几乎不手写链表

生产代码中,Python 的 `list` 覆盖了绝大多数"表"的需求: 随机访问 $O(1)$ 、尾部插入 $O(1)$ 、缓存友好,常数小到足以抵消链表的 $O(1)$ 定位插入优势; 需要头尾都 $O(1)$ 时用 `collections.deque` (内部是双向链接的块,专门优化两端)。**手写链表的用武之地:** ① 面试考察指针思维; ② 实现树/散列桶等"必须显式引用"的结构(第4、5章); ③ 需要"多指针共享结点"的特殊场景(如并查集的可持久化)。判断标准: 凡是能按下标访问需求的,一律 `list`。

§3.5 栈

3.5.1 栈 ADT 与 list 实现

栈(stack): 后进先出(LIFO)。三个操作——`push`(压入)、`pop`(弹出)、`top`(查看栈顶)。数组实现最简单: 只在尾部增删,三个操作全部 $O(1)$ 。Python 里 `list` 天然就是数组栈: `append` 即 `push`, `pop()` 即 `pop`, `a[-1]` 即 `top`。

示例 3-5 用 list 模拟栈(与 C 数组栈对照)

```
# C:  int st[100]; int top = 0;
#     st[top++] = x;           // push
#     x = st[--top];           // pop
st = []                        # 空栈
st.append(3)                   # push:3
st.append(1)                   # 3 1
st.pop()                       # pop:移除 1
print(st[-1])                  # top:3(空栈时 st[-1] 抛 IndexError,是好事)
print(len(st) == 0)            # 判空
```

注意与 C 的函数调用栈对照: 递归深度过大时抛 `RecursionError`, 本质就是这同一个结构——调用栈被 `push` 到溢出(第1章讲过)。栈也可以用单链表实现(表头做栈顶,头插头删 $O(1)$), 接口完全不变——这正是 §3.1 说的 ADT 威力。

3.5.2 应用一:括号平衡检查

示例 3-6 括号平衡:三种括号匹配检查

```
def is_balanced(s):
    """判断字符串中 ( ) [ ] { } 是否配对且顺序正确。"""
    st = []
    for c in s:
        if c in "([{":
            st.append(c)          # 左括号入栈
        elif c in ")]}":
            if not st:
                return False     # 右括号多余
            top = st.pop()
            if (c == ")" and top != "(") or \
                (c == "]" and top != "[") or \
                (c == "}" and top != "{"):
                return False     # 类型不匹配
    return not st                # 结束时栈空 = 左括号无剩余
```

3.5.3 应用二:后缀表达式求值

后缀表达式(逆波兰记法)把运算符放在操作数之后: `6 5 2 3 + 8 * + 3 + *`, 不需要括号。求值规则:遇数字入栈;遇运算符弹出两个操作数,算完把结果压回。这个"惰性计算"模式是栈最漂亮的用法。

示例 3-7 后缀表达式求值

```
def eval_postfix(toks):
    """输入 token 列表(数字串与运算符);输出表达式值。"""
    st = []
    for t in toks:
        if t.lstrip("-").isdigit():
            st.append(int(t))      # 数字入栈
        else:
            b = st.pop()           # 先弹的是右操作数!
            a = st.pop()
            if t == "+":
                st.append(a + b)
            elif t == "-":
                st.append(a - b)
            elif t == "*":
                st.append(a * b)
            else:
                st.append(a // b)  # 整除
    return st.pop()              # 最后栈里只剩结果

print(eval_postfix(["6", "5", "2", "3", "+", "*", "+"])) # 31
```

⚠ 误区 2:操作数弹出顺序

后缀 `8 2 /` 是 $8/2=4$,但弹栈顺序是先弹 2 再弹 8——先弹的是右操作数。减法和除法对顺序敏感,写反了就是 $2/8$ 。口诀:栈顶是"后出现"的操作数,后出现的是右操作数。

3.5.4 应用三:中缀转后缀

人读的是中缀(`a+b*c`),机器算的是后缀。转换算法(调度场算法):遇数字直接输出;遇运算符,先把栈中**优先级不低于它的**运算符弹出输出,再压入自己;左括号直接入栈,右括号则弹出到左括号为止。这样保证高优先级先输出。

示例 3-8 中缀转后缀(单字符数字简化版)

```
def prec(op):
    return 1 if op in "+-" else 2

def infix_to_postfix(s):
    """输入如 "6+5*(2+3)"(无空格),输出后缀字符串。"""
    out = []
    st = []
    for c in s:
        if c.isdigit():
            out.append(c)          # 数字直接输出
        elif c == "(":
            st.append(c)
        elif c == ")":
            while st[-1] != "(":
                out.append(st.pop())
            st.pop()              # 丢弃左括号
        else:                    # 运算符
            while st and st[-1] != "(" and prec(st[-1]) >= prec(c):
                out.append(st.pop())  # 栈顶优先级不低于当前则弹出
            st.append(c)
    while st:
        out.append(st.pop())
    return "".join(out)

print(infix_to_postfix("6+5*(2+3)"))  # 6523+*+
```

例题 3-1 手工走一遍中缀转后缀

输入 $6+5*(2+3)$:6 输出;+ 入栈;5 输出;* 入栈(+ 优先级低,留在栈中);(入栈;2 输出;+ 入栈;3 输出;) 弹出 +,再弹(; 结束弹栈得 *、+。输出: $6523+*+$ 。用 §3.5.3 求值: $2+3=5, 5*5=25, 6+25=31$ 。✓

§3.6 队列

3.6.1 队列 ADT 与循环数组实现

队列(queue):先进先出(FIFO)。enqueue(入队)在队尾,dequeue(出队)在队头。用数组实现有个经典问题:出队后前面空出的位置怎么办?如果每次出队都把全部元素前移,那是 $O(n)$;不移动、让队头指针"追赶"队尾指针,数组就要**循环**使用——下标到底后绕回 0。

示例 3-9 循环数组队列(用 count 记录元素数)


```

# front 指向队首元素,back 指向队尾下一个空位,count 记录元素数
class MyQueue:
    def __init__(self, cap):
        self._data = [None] * cap
        self._front = 0
        self._back = 0
        self._count = 0

    def push(self, x):          # 入队:假设未满(满的处理见误区3)
        self._data[self._back] = x
        self._back = (self._back + 1) % len(self._data)
        self._count += 1

    def pop(self):             # 出队
        x = self._data[self._front]
        self._front = (self._front + 1) % len(self._data)
        self._count -= 1
        return x

    def front(self):
        return self._data[self._front]

    def is_empty(self):
        return self._count == 0

    def __len__(self):
        return self._count

```

★ 重点:空/满判断的两种方案

若用 front 与 back 两个下标判断空满,会遇到 front == back 时"既可能空也可能满"的歧义。两种解法:① **计数法**(示例 3-9,多存一个 count,最简单);② **浪费一格法**(规定 back 永远不追赶 front,队满时留一个空格,满条件是 (back+1) % cap == front)。工程与面试都常考,两种都要会。

3.6.2 标准库:collections.deque

示例 3-10 deque:两头都 O(1) 的双端队列

```

from collections import deque

q = deque()          # 空队列
q.append(1)          # 右端入队
q.append(2)
q.append(3)
while q:
    print(q.popleft(), end=" ")  # 左端出队:1 2 3(先进先出)

d = deque([1, 2, 3])
d.appendleft(0)      # 左端入队:0 1 2 3
d.append(4)          # 右端入队:0 1 2 3 4
print(list(d))       # [0, 1, 2, 3, 4]

```

C 的手写循环队列	Python 的标准库
自己维护 front/back 下标与取模	<code>q.append / q.popleft()</code> 一行一个操作
空/满判断要写计数或浪费一格	自动扩容,永不"满"
<code>free</code> 忘记调用即泄漏	垃圾回收器自动释放

⚠ 误区 3:循环队列的下标溢出与"满"

示例 3-9 简化了"队满"检查——满时再 `push` 会覆盖旧元素。生产实现必须显式处理:计数法判 `count == cap` 满则拒绝/扩容,浪费一格法判 `(back+1) % cap == front`。另一个易错点是取模:back 先加 1 再对 capacity 取模,而不是对"当前元素数"取模。工程中直接用 `deque` 即可绕开全部这些问题。

§3.7 选型与工程实践

3.7.1 数组 or 链表?—一张决策表

场景	选择	理由
按序访问所有元素(遍历)	list	连续内存,缓存友好,常数小
大量随机下标访问	list	$O(1)$ vs $O(n)$
只做尾部增删(栈)	list	均摊 $O(1)$,实现最简单
两端增删(队列)	deque / 循环数组	两头都 $O(1)$
需要多引用共享结点	手写链表/树	list 做不到引用语义的共享
几乎一切其他场景	list	Python 社区共识:先 list,不够再换

3.7.2 栈模拟递归与调试技巧

任何递归都可以改写为"显式栈 + 循环"的迭代版(第4章树遍历会用到):函数调用栈压的是"返回地址 + 局部变量",显式栈压的是"待处理的任务"。迭代版不受解释器递归深度限制(默认约 1000 层),深度优先搜索(第9章)的大图遍历通常必须用迭代版。调试栈程序的两个技巧:① `push` 前打印"压入 x",`pop` 后打印"弹出 x",肉眼跟踪栈的演化;② 每次 `pop` 前检查栈非空,把"逻辑错误"提前变成"崩溃点",定位速度翻倍。

🚀 工程建议:优先内建结构,手写只为理解

本章手写了动态数组、链表、栈、队列四样东西——目的是让你看清每一层的复杂度来源。生产代码请一律使用 `list / deque`:它们是 C 实现的内建类型,速度比任何纯 Python 手写都快一个数量级,边界处理也更健壮。手写的用武之地是"内建没有的结构"(后几章的树、堆、散列表、图),那才是本书后续的主战场。

本章复杂度速查表

操作	数据结构 / 算法	平均	最坏
按下标访问	list(动态数组)	$O(1)$	$O(1)$
尾部插入(append)	list	$O(1)$	$O(n)$ (扩容)
头部插入(insert(0))	list	$O(n)$	$O(n)$
任意位置插入/删除	list	$O(n)$	$O(n)$
按值查找	list	$O(n)$	$O(n)$
切片 a[i:j]	list	$O(j-i)$	$O(j-i)$
按下标访问	链表	$O(n)$	$O(n)$
按值查找	链表	$O(n)$	$O(n)$
头部/尾部插入	链表	$O(1)$	$O(1)$
已定位处插入/删除	链表	$O(1)$	$O(1)$
push / pop / top	栈(list 或链表实现)	$O(1)$	$O(1)$
入队 / 出队 / 取队头	循环数组队列	$O(1)$	$O(1)$
两端增删	deque(双端队列)	$O(1)$	$O(1)$
括号平衡检查	栈扫描一遍	$O(n)$	$O(n)$
后缀表达式求值	栈扫描一遍	$O(n)$	$O(n)$
中缀转后缀	调度场算法	$O(n)$	$O(n)$
list 扩容	n 次 append 总代价	摊还 $O(n)$, 平均每次 $O(1)$ (第11章证明)	

最小必会清单

- 能独立说出 ADT 的定义,并举例说明"接口与实现分离"的好处
- 能独立解释 list 的 append 为什么平均 $O(1)$,扩容时发生了什么
- 能独立说出 `[[]] * 5` 与 `[[] for _ in range(5)]` 的区别
- 能独立默写单链表的表头插入、查找、删除(含删表头特例)
- 能独立说出 Python 中"几乎不手写链表"的理由,以及仍需手写的三种场景
- 能独立用 list 模拟栈,并解决括号平衡问题
- 能独立写出后缀表达式求值,并说清操作数的弹出顺序
- 能独立默写中缀转后缀算法(含运算符优先级处理)
- 能独立实现循环数组队列,并说出空/满判断的两种方案
- 能独立说出 list 与 deque 的选型规则(至少 4 条场景)

自测题

Q 1. list 的 append 为什么说"平均 $O(1)$ "而最坏 $O(n)$?

✓ 参考答案

平时插入只写一个元素 $O(1)$; 仅当长度达到容量时才触发扩容, 需拷贝全部 n 个元素的引用 $O(n)$ 。容量按比例倍增, 从 1 增到 n 共 $\log_2 n$ 次扩容, 总拷贝 $n+n/2+\dots\approx 2n$, 摊到 n 次插入上平均 $O(1)$ 。

Q 2. `a = []`; `a[0] = 1` 会怎样? 与 C 的对应写法有何不同?

✓ 参考答案

抛 `IndexError` —— 下标赋值不会自动扩容。C 里 `int a[0]` 不合法但 `int *p = malloc(0); p[0] = 1` 是未定义行为(可能静默破坏内存)。Python 显式报错, 把 UB 变成可定位的异常, 这是 Python 更好调试的原因之一。

Q 3. 单链表中删除“值为 x 的结点”, 为什么表头要单独处理? 怎样统一?

✓ 参考答案

普通结点删除需要前驱结点改指针, 而表头没有前驱。解法: ① 特判(本章示例 3-4); ② 引入哑结点(dummy), 表头前面永远站一个空结点, 删除逻辑统一; ③ 让 `erase` 返回新表头(示例 3-4 的做法), 把“改表头”交给调用方。哑结点法在工程中最常用。

Q 4. 后缀表达式 `8 2 / 3 5 * +` 的值是多少? 写出求值过程。

✓ 参考答案

8、2 入栈; `/`: 弹 2、8, $8/2=4$ 入栈; 3、5 入栈; `*`: 弹 5、3, $3*5=15$ 入栈; `+`: 弹 15、4, $4+15=19$ 。答案 19。注意先弹的是右操作数。

Q 5. 循环队列用 `front` 与 `back` 两个下标时, 为什么 `front == back` 会“既可能空也可能满”? 两种解法是什么?

✓ 参考答案

队列空时 `front` 与 `back` 重合; 而队满时 `back` 绕一圈也追上 `front`, 两个状态无法区分。解法: ① 计数法——额外维护元素个数 `count`, 空满由 `count` 判断; ② 浪费一格法——规定 `back` 永不与 `front` 重合, 满条件是 $(back+1) \% cap == front$, 牺牲一个格子换取 $O(1)$ 判断。

Q 6. 中缀 `a+b*c-d` 转后缀的过程是什么? 为什么 `*` 在 `+` 之前输出?

✓ 参考答案

`a` 输出; `+` 入栈; `b` 输出; 遇 `*`: 栈顶 `+` 优先级(1) 低于 `*`(2), 不弹出, `*` 入栈; `c` 输出; 遇 `-`: 栈顶 `*` 优先级不低于 `-`, 弹出 `*` 输出; 栈顶 `+` 优先级不低于 `-`, 弹出 `+` 输出; `-` 入栈; `d` 输出; 结束弹栈输出 `-`。结果 `abc*+d-`。`*` 先输出是因为它绑定得更紧(优先级高), 后缀表达式中运算符顺序即计算顺序。

章末实验题:中缀表达式计算器

实验 3:中缀表达式计算器(栈应用综合)

实验目标:实现一个命令行计算器:读入含 $+$ $-$ $*$ $/$ 与括号的中缀表达式,支持多位数,输出后缀表达式与计算结果。

实验要求:

- 任务 1:实现 tokenize,把字符串拆成数字与运算符序列,支持多位数(知识点:\$3.5.4 中缀转后缀的扩展)
- 任务 2:实现中缀转后缀,处理括号与优先级(知识点:\$3.5.4 调度场算法)
- 任务 3:实现后缀求值,注意弹出顺序(知识点:\$3.5.3)
- 任务 4:主循环读入表达式,输出后缀式与结果,输入 `quit` 退出(知识点:\$3.5 栈应用综合)
- 任务 5:处理非法输入:除数为 0、括号不匹配时给出错误信息而非崩溃(知识点:\$3.5.2 括号平衡)

实验环境:Python 3.8+,标准库即可。运行: `python lab3.py`。

第一步 tokenize 用指针扫描:连续数字拼成一个字符串 token;空格跳过;其余字符单独成 token

第二步 中缀转后缀直接复用示例 3-8 的骨架,把"单字符数字"换成"字符串 token";数字 token 用 `t.lstrip("-").isdigit()` 判定

第三步 求值时先检查操作数栈是否够两个元素(非法输入),再检查除数是否为 0;异常用 `raise ValueError` 抛出,主循环统一捕获

第四步 括号匹配错误在转换阶段发现:弹栈遇空、或结束后栈里剩左括号,都报"括号不匹配"

✅ 参考答案(完整可运行程序,约 85 行,分两段)

```
# lab3.py — 中缀表达式计算器(第一段:词法与转换)
def tokenize(s):
    """任务 1:拆分 token(支持多位数与空格)。"""
    toks = []
    i = 0
    while i < len(s):
        if s[i].isspace():
            i += 1
            continue
        if s[i].isdigit():
            j = i
            while j < len(s) and s[j].isdigit():
                j += 1
            toks.append(s[i:j])
            i = j
        else:
            toks.append(s[i])
            i += 1
    return toks

def prec(op):
    return 1 if op in "+-" else 2

def infix_to_postfix(toks):
    """任务 2:中缀转后缀(调度场算法)。"""
    out = []
    st = []
    for t in toks:
        if t.lstrip("-").isdigit():
            out.append(t)                # 数字直接输出
        elif t == "(":
            st.append(t)
        elif t == ")":
            while st and st[-1] != "(":
                out.append(st.pop())
            if not st:
                raise ValueError("括号不匹配")
            st.pop()                    # 丢弃左括号
        else:
            # 运算符
            while st and st[-1] != "(" and prec(st[-1]) >= prec(t):
                out.append(st.pop())
            st.append(t)
    while st:
        if st[-1] == "(":
            raise ValueError("括号不匹配")
        out.append(st.pop())
    return out
```

lab3.py(第二段:求值与主循环)

```

def eval_postfix(toks):
    """任务 3:后缀求值(注意弹出顺序)。"""
    st = []
    for t in toks:
        if t.lstrip("-").isdigit():
            st.append(int(t))
        else:
            if len(st) < 2:
                raise ValueError("操作数不足")
            b = st.pop()          # 先弹的是右操作数
            a = st.pop()
            if t == "+":
                st.append(a + b)
            elif t == "-":
                st.append(a - b)
            elif t == "*":
                st.append(a * b)
            else:
                if b == 0:
                    raise ValueError("除数为 0")
                st.append(a // b)
    if len(st) != 1:
        raise ValueError("表达式非法")
    return st[0]

def main():
    print("中缀表达式计算器(支持 + - * / 与括号,输入 quit 退出)")
    while True:
        line = input("> ").strip()
        if line == "quit":
            break
        try:
            toks = tokenize(line)
            post = infix_to_postfix(toks)
            result = eval_postfix(post)
            print("后缀式:", " ".join(post))
            print("结果:", result)
        except ValueError as e:
            print("错误:", e)

if __name__ == "__main__":
    main()

```

示例输入输出:


```
> 6+5*(2+3)
后缀式: 6 5 2 3 + * +
结果: 31
> 12/4+7*2
后缀式: 12 4 / 7 2 * +
结果: 17
> (1+2)*3
后缀式: 1 2 + 3 *
结果: 9
> 5/0
错误: 除数为 0
> (1+2
错误: 括号不匹配
> quit
```

设计说明:① tokenize/转换/求值三阶段分离,每阶段只做一件事,出错信息定位到阶段——这是编译原理"词法→语法→语义"流水线的雏形;② 转换与求值都基于同一个栈结构(list 模拟),体现了 ADT"实现替换、接口不变"的思想;③ 错误用 `ValueError` 统一抛出,主循环一个 `except` 兜底,避免每个函数各自打印错误导致"错误处理与逻辑纠缠";④ 运算符优先级集中在 `prec` 一个函数里,新增运算符(如幂 `**`)只需改这一处。

下一章预告:第4章 树——把"链式结点"推广为"有分支的结构",学习二叉树遍历、表达式树、二叉查找树与 AVL 平衡。第4章的表达式树与本章的中缀转后缀直接衔接;树的层序遍历将使用本章的队列。